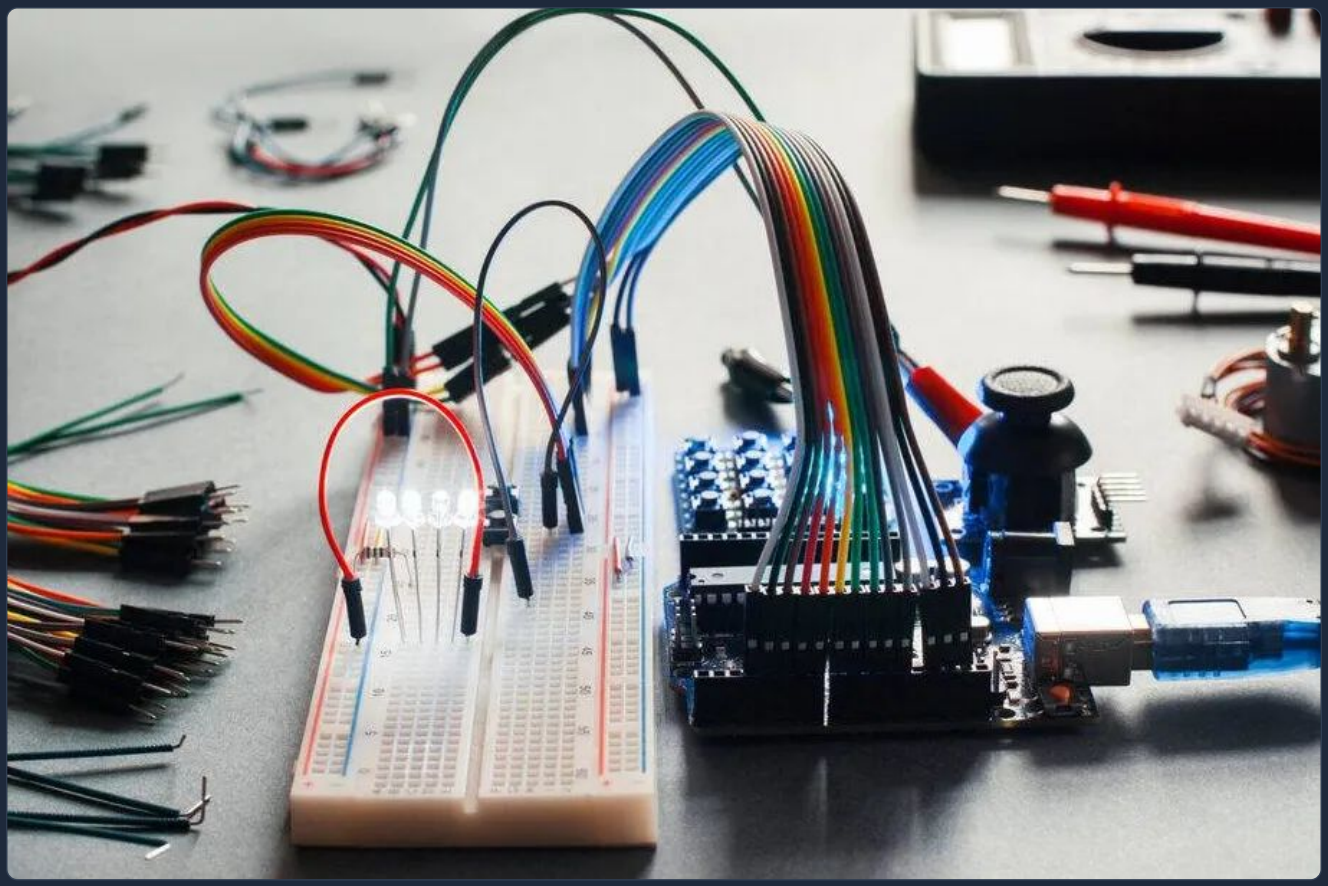
8051 HARDWARE LAB · PLAIN-ENGLISH GUIDE

How the Software and the Hardware Work Together

An AT89S52 microcontroller board, explained from the outside in: what the parts are, what the code says, where the two meet, and how to talk about it.



BUILT BY

Atharva (24BEL0082)
Aditya (24BEL0087)

DEPARTMENT

Electronics and Computer Engineering
AY 2025-26

INSIDE

18 pages · real photographs
13 programs · 80 parts
one shared data bus

The whole project in one paragraph

START HERE

This is a small computer board that watches a room and reacts to it. At the centre sits a single chip, an Atmel **AT89S52** microcontroller, which is a complete computer no bigger than a thumb: a processor, 8 kilobytes of program memory and 256 bytes of working memory, all inside one black plastic block with forty metal legs. We wired real parts around it: eight indicator LEDs, a two-digit seven-segment display, a 16-character LCD, a temperature sensor, a relay, a buzzer, a bank of eight switches, and a cable that lets the board type lines of text onto a PC screen. Then we wrote thirteen small programs in Keil C51, compiled each one into a machine-code `.hex` file, and burned those files into the chip. The board now reads its switches and its sensor, works out what they mean, and moves its outputs: light an LED, print on the LCD, click the relay, send a line of telemetry upstairs. Everything it does is that same loop, over and over, hundreds of times a second.

If you remember only three things

1. **The chip ran out of pins, and we did not add a chip to fix it.** The finished design needs 54 separate connections. The processor has 32 to spare. Instead of buying an expander we made eight of those pins carry data for four different devices, one at a time, and used four more pins as gates that decide which device is listening.
2. **Software is a list of instructions that change the voltage on a pin.** Nothing more mystical than that. When the code says the relay is on, five volts appear on pin 14 of the chip, and a millisecond later the relay clicks. The software never touches the world directly; it only ever moves pins.
3. **Every program here runs the same four beats: read, convert, decide, act.** The largest and most complicated program on the board is that loop with a longer decide step. Once you can say those four beats out loud, you can explain all thirteen programs.

> **HOW TO READ THIS GUIDE**
>
> Pages 3 to 6 explain the two halves separately, first the hardware you can touch and then the code you cannot. Pages 7 to 10 explain how they join and how we solved the shortage of pins. Pages 11 to 14 walk through the physical build and then follow two real events, a button press and a rise in temperature, all the way through the system. Pages 15 to 18 are reference: the tests, the numbers, a glossary, and a short script you can use when you are asked to present the work.

Two halves of one machine

CONCEPT

Every embedded system has these two halves, and it is worth being blunt about the difference. One half you can drop on the floor. The other half you cannot hold at all: it is a pattern of electrical charges inside the chip. The entire job of this project is joining them.

Hardware — the things you can touch

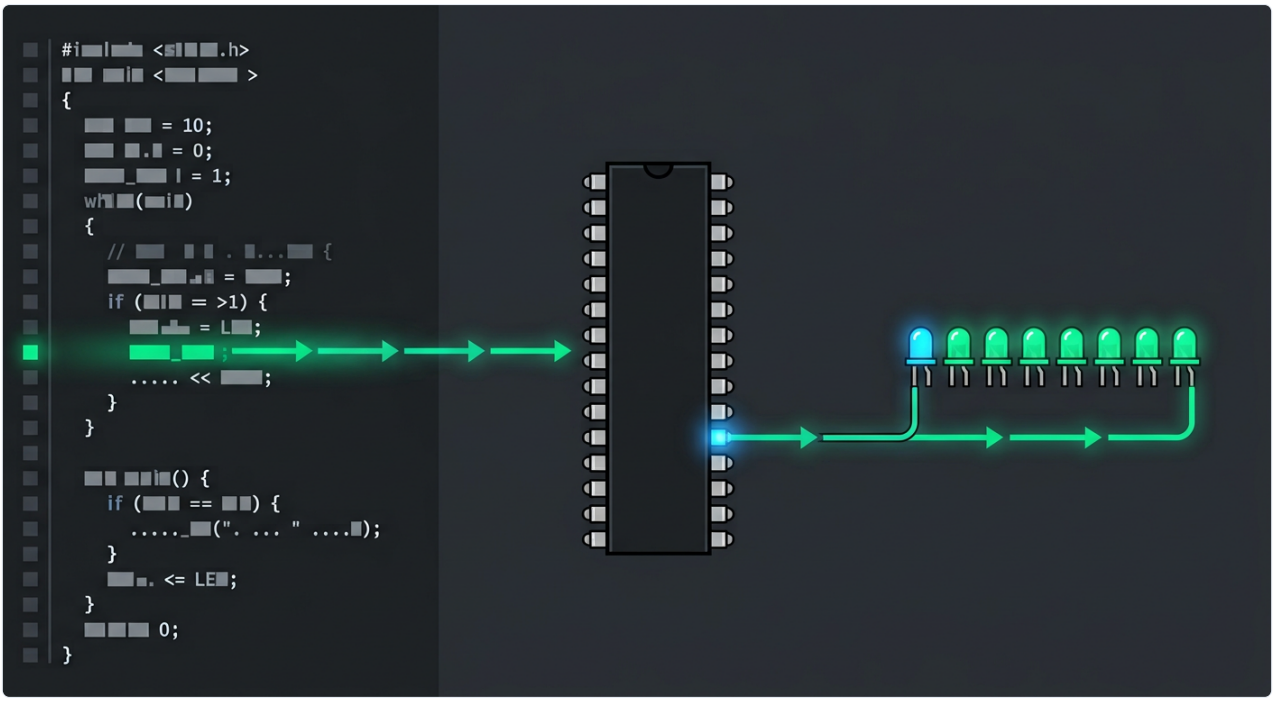
The black chip, the eight LEDs, the display, the LCD, the temperature sensor, the relay, the buzzer, the resistors, the wire. Hardware is physical and dumb. Left alone it does nothing at all: it is a collection of parts waiting for electricity to arrive in the right order.

It also sets the limits. A pin can only supply so much current. A sensor can only measure so accurately. An LED needs a resistor or it burns out. Most of the engineering on this board was respecting those limits.

Software — the list of instructions

Thirteen programs written in C, typed into the Keil uVision editor and compiled into `.hex` files of bare machine bytes. Burn those bytes into the chip's flash memory and they survive with the power off. Switch on and the chip starts at the first instruction and obeys them one at a time, forever.

Software has no physical form. Change one character and recompile and you have a completely different machine, with no change to the hardware at all. That is the whole reason these boards are useful.



The boundary between the two halves is a single pin. One line of code becomes one electrical signal on one pin, and that signal becomes light from one LED. Nothing else happens anywhere in the machine.

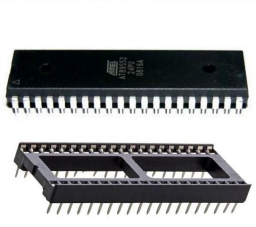
Follow one instruction all the way out

1. **In the code:** the program calls `led_write(0xFF)`. The compiler has already turned that line into three or four machine instructions stored in flash.
2. **In the chip:** the processor fetches those instructions and executes them. Eight pins of port 0 are driven to 5 V at once, then a ninth pin, the LED latch gate, is pulsed high for a fraction of a microsecond.
3. **In the hardware:** the latch chip captures that pattern and holds it. Current flows through a 470 ohm resistor into each LED, about 7 milliamps each, and all eight light up. They stay lit after the bus has moved on to something else.

The parts you can touch

HARDWARE ONE

These are the six parts that do the visible work: they are what a person looking at the board actually notices. Each one is wired to the chip and each one is driven by a line of code somewhere in the thirteen programs.



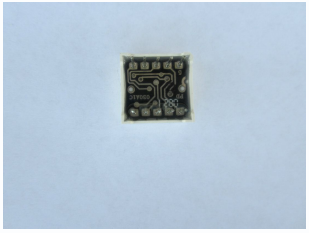
AT89S52 microcontroller U1

The brain. Forty pins, 8 KB of flash for the program, 256 bytes of RAM for working values, three timers and a serial port, all in one package. Runs at 11.0592 MHz.



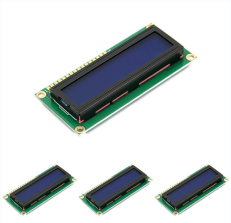
Eight indicator LEDs D3-D10

Six of them carry meaning: safe, warning, danger, relay, buzzer and a heartbeat that blinks once a second to prove the chip is alive. Each has its own 470 ohm resistor.



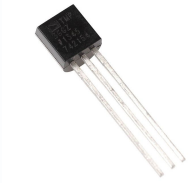
Two-digit display DSP1

Shows the temperature in whole degrees. Only one digit is ever lit at a time; the code swaps between them every 5 ms, which is fast enough that the eye sees both.



16x2 character LCD LCD1

Two lines of sixteen characters. Line one carries the live reading, line two carries the state: SAFE, WARNING, DANGER, or EMERGENCY STOP.



LM35 temperature sensor U6

The thermometer. It needs no calibration and puts out exactly 10 millivolts per degree Celsius, so 0.31 V on its leg means 31 degrees. Three legs, looks like a small transistor.



Relay RLY1

An electrically operated switch. Five volts on one pin from the chip lets it switch a load a hundred times bigger, such as a fan or a pump. You hear it as a click.

A DETAIL WORTH SAYING OUT LOUD

Notice that five of these six parts need eight wires each. That single fact is what forced the design decision described on page 8. It is the only genuinely difficult thing about this board, and it is the part most worth understanding before a viva.

The parts that make the rest work

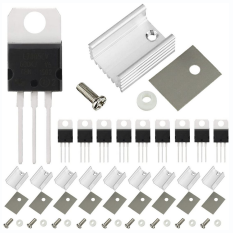
HARDWARE TWO

None of these parts do anything a person would notice, and the board is dead without every one of them. This is the half of the hardware that students forget to explain, and examiners notice.



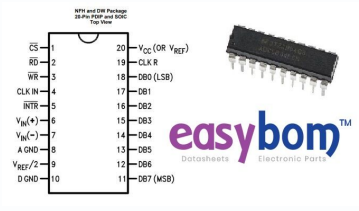
11.0592 MHz crystal `Y1`

The metronome. Every instruction, every delay and the serial speed all come from this. The odd frequency is chosen on purpose: it divides exactly by 9600, so serial communication has zero timing error.



7805 voltage regulator `U7`

Takes the 9 V from the adapter and holds a rock-steady 5 V no matter what the board is doing. Everything else on the board runs on that 5 V. It gets warm, so it wears a heatsink.



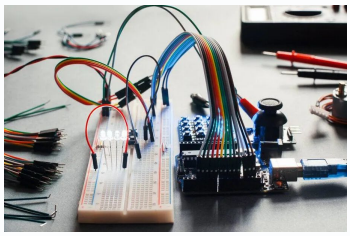
ADC0804 converter `U4`

The translator. The sensor speaks in voltages, but the chip only understands numbers. This chip measures a voltage and hands back a whole number from 0 to 255, about 114 microseconds per reading.



USBasp programmer `P1`

How the code gets in. It plugs into the PC's USB port and writes the compiled .hex file straight into the chip's flash memory. The board can be reprogrammed hundreds of times.



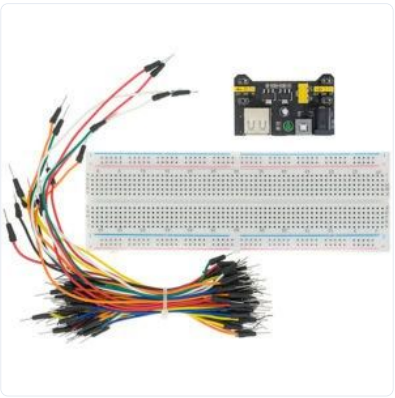
Solderless breadboard `—`

The base. A plastic block full of spring contacts that lets parts be connected with push-in wire and no soldering at all, so a mistake costs ten seconds to undo.



Eight-way DIP switch `SW1`

Eight tiny sliding switches standing in for real industrial inputs: a door sensor, a motor running, a tank level. Lets the board be tested without any real machinery attached.



The board under construction on the breadboard. Every part is through-hole and hand-placeable: no reflow oven, no microscope, no printed circuit board.

The code: thirteen small programs

SOFTWARE

The software is not one large program. It is thirteen small ones, each building on the one before, each small enough to read on a single screen. Write a program, compile it, burn it, watch the hardware prove it works, then move to the next. That order is why this project is easy to demonstrate: at every stage something on the board is doing something visible.

#	Program	What it does, in plain English	Hardware it drives
01	led_blink	Flashes the eight LEDs once a second. Proves the chip, the clock and the output wiring all work.	LED bank
02	led_patterns	Walks a lit LED left and right and counts up in binary, so a stuck bit would be obvious.	LED bank
03	switch_led	Reads the eight switches and lights the matching LED. The first program with a real input.	LEDs, switches
04	seg7_counter	Counts from 00 to 99 on the display, driven by a timer interrupt rather than a delay loop.	7-segment
05	timer0_second	Blinks one LED exactly once a second using the hardware timer, which is far more accurate than counting instructions.	Timer 0, LEDs
06	square_wave	Produces a 1.00 kHz square wave on a pin, checked on an oscilloscope against a calculated reload value.	Timer 0, P3.4
07	ext_interrupt	Press a button and the chip drops whatever it is doing immediately and responds. No waiting, no polling.	INT0, INT1, LEDs
08	uart_echo	Sends back every character the PC types. Proves the serial link in both directions.	UART, MAX232
09	uart_commands	A small command shell: type LED FF and all eight LEDs obey; type RLY 1 and the relay clicks.	UART, LEDs, relay
10	adc_7seg	Reads the temperature and shows it as two digits on the display.	ADC, LM35, display
11	adc_lcd	Shows the temperature on the LCD as a number and as a bar of solid blocks that grows with heat.	ADC, LM35, LCD
12	industrial_monitor	The finished system. Reads the temperature, colours the zone, drives relay and buzzer, and prints one line of telemetry every second.	Everything
13	selftest	Runs a check of every part on power-up and prints PASS or the name of the fault, so a dead board is diagnosed in seconds.	Everything

> **ONE LINE TO REMEMBER**
>
> The largest of these thirteen programs compiles to 3.4 kilobytes, which is well under half of the 8 KB of flash in the chip, and uses 30 bytes of the 256 available. The constraint on this board was never memory. It was pins — see page 8.

Where the software touches the metal

THE JOIN

The chip has forty legs. Thirty-two of them are general-purpose input/output pins, arranged in four groups of eight called **P0**, **P1**, **P2** and **P3**. Every single thing the software does to the outside world is done by driving one of those thirty-two pins to 5 V or to 0 V, or by reading which of the two it currently is.

Pin	Name in the code	What happens when the code drives it
P0.0 – P0.7	BUS	The eight shared data lines. Whatever number is on the bus right now is visible to the LEDs, the display, the LCD and the ADC all at once.
P1.0 – P1.7	SWITCH	Reads the eight DIP switches. A closed switch pulls the pin to 0 V.
P2.0	LED_LE	Pulsed high to open the LED latch, which grabs the number on the bus and holds it.
P2.1	SEG_LE	Pulsed high to open the display latch, same idea for the seven-segment digits.
P2.2	LCD_E	Tells the LCD that the byte sitting on the bus is ready to be accepted.
P2.3	LCD_RS	Selects whether that byte is a command, such as clear the screen, or a character to print.
P2.5 / P2.6	ADC_WR / ADC_RD	Pulses that tell the converter to start a reading, then to hand the answer over.
P2.7	ADC_INTR	Read, not driven. The converter pulls this low when the answer is ready.
P3.2	PB_EMERG	The emergency stop button, wired to interrupt 0 so it is answered immediately.
P3.3	PB_MODE	The reset button, wired to interrupt 1, clears the emergency state.
P3.4	RELAY	5 V here energises the relay coil. The click is this pin changing state.
P3.5	BUZZER	5 V here sounds the buzzer.
P3.6 / P3.7	DIG1 / DIG2	Chooses which of the two digits is lit at this instant.

And in the code itself, that is this

```
void bus_put(unsigned char v)
{
    bus_shadow = v;
    BUS = v;
}
```

That is the whole trick. `bus_put()` remembers the number in a variable and pushes it out onto the eight pins of port 0. Then one more line pulses a gate pin, and whichever device owns that gate takes the number and keeps it. Everything else in the thirteen programs is built on those two moves.

The problem: 54 wires, 32 pins

CONSTRAINT

Here is the difficulty that shaped the entire design, and the one question worth being ready for. Count the connections the finished board needs, then count the pins the chip has.

What needs connecting	Wires it would want
LED bank, eight lights	8
Seven-segment display, seven segments plus the decimal point	8
Character LCD, its data lines	8
Analogue-to-digital converter, its data lines	8
DIP switch bank, eight inputs	8
Subtotal for data alone, with every device private	40
Control and strobe lines: two latch gates, three LCD lines, three converter lines	8
Relay, buzzer, two buttons, two digit selects	6
Total the design needs	**54**
Free input/output pins on the chip, P0 to P3	**32**
Shortfall	**22**

Three ways out, and why we chose the third

Option	What it costs	Verdict
Buy a bigger chip, such as one with 60 pins	A new part, a new package, a printed circuit board, a reflow process, and it stops being a breadboard project.	Rejected
Add port expander chips on a serial bus	Two extra ICs, roughly 250 rupees, and every output update becomes several slow serial transactions.	Rejected
Share one set of eight pins between the four data devices, with a gate per device	Free. Costs a few microseconds per update and demands disciplined code.	**Chosen**

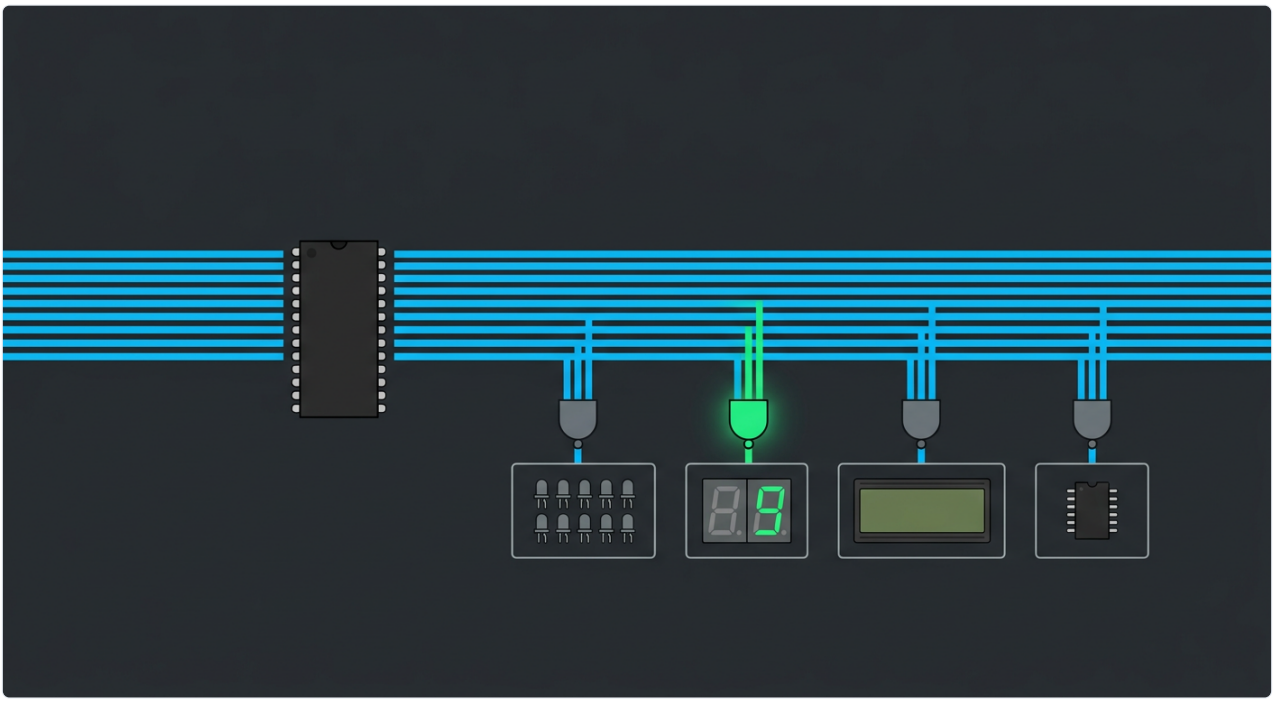
THE ONE SENTENCE

The board could not afford 54 private wires, so the eight lines of port 0 became a single shared road, and four spare pins became the gates that decide who is listening. That decision is the project.

The fix: one road, many gates

ARCHITECTURE

A latch is a chip that copies the eight numbers on its input and keeps holding them on its output until told to change. Put one latch in front of each device and a fascinating thing becomes possible: all four devices can watch the same eight wires, and only the one whose gate is opened takes any notice.



One eight-line bus leaves the chip and runs past four devices. Each device has its own gate. Exactly one gate is open at any moment, shown here in green; the other three are closed and ignore whatever is on the road.

Writing to a device takes four moves

1. Put the number on the shared bus: `bus_put(0x55)`. All four devices see it, and all four ignore it, because their gates are shut.
2. Pulse the gate of the device you actually want: `LED_LE = 1` then immediately `LED_LE = 0`.
3. The latch behind that gate captures the number in that instant and goes back to ignoring the bus.
4. Move on. The LEDs stay showing 0x55 while the bus runs off to update the display, the LCD or anything else.

Device	Its gate pin	Called in the code
LED bank	P2.0	LED_LE
Seven-segment display	P2.1	SEG_LE
Character LCD	P2.2	LCD_E
Analogue-to-digital converter	P2.5 and P2.6	ADC_WR and ADC_RD

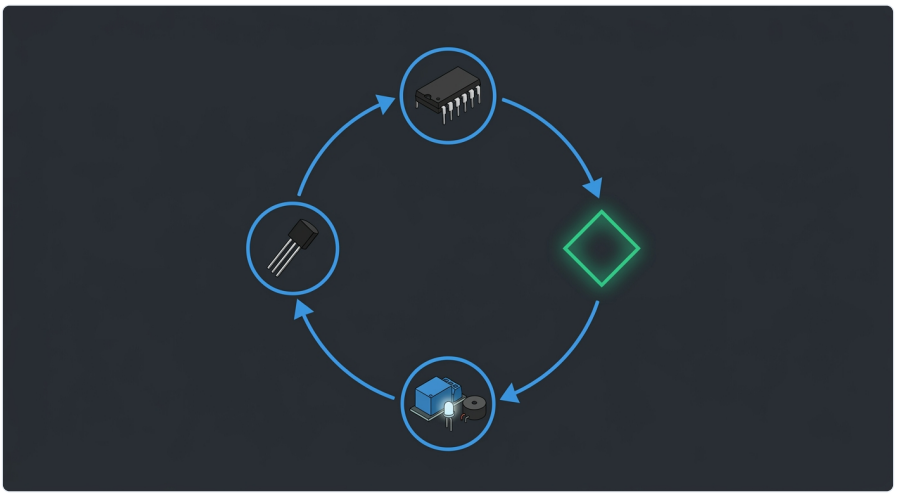
WHAT SHARING COSTS, HONESTLY

Two things. First, speed: four devices can no longer be updated at once, only one after another. At a microsecond per device that is irrelevant here. Second, discipline: the code must always know which device last had its gate open, and must never leave two gates open together. That is why every output goes through one small set of functions in `board.h` rather than being written directly.

The loop: read, convert, decide, act

BEHAVIOUR

Once the wiring problem is solved, the behaviour is simple. Every program on this board, including the largest one, is the same four beats repeated forever. Program 12, the finished industrial monitor, is the clearest example.



The four beats, clockwise: the sensor measures, the converter turns the measurement into a number, the code decides what that number means, and the outputs move. Then it starts again.

1. **Read.** Every five milliseconds a timer interrupt fires and refreshes one digit of the display. Every time the main loop comes round, the code asks the converter for a temperature reading and the switches for their positions.
2. **Convert.** The converter returns a whole number from 0 to 255. The code multiplies it by 250 and divides by 32, which turns it into tenths of a degree. So a reading of 128 means 40.0 degrees.
3. **Decide.** Below 40.0 degrees is the safe zone. From 40.0 to 69.9 is the warning zone. At 70.0 and above is the danger zone. The emergency button overrides all three.
4. **Act.** Write the LED pattern, refresh a digit, update the two LCD lines, and once every second print a full line of telemetry to the PC. Then start again.

Zone	Temperature	LEDs lit	Relay	Buzzer	Second LCD line
SAFE	below 40.0 °C	green	off	off	SAFE FAN:OFF
WARNING	40.0 to 69.9 °C	yellow	on	off	WARNING FAN:ON
DANGER	70.0 °C and above	red	on	on	DANGER FAN:ON
EMERGENCY	any temperature	red, flashing	off	on	EMERGENCY STOP

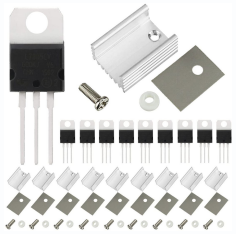
The sixth LED, the heartbeat, toggles once per second independently of all of this. It is there so that anyone can tell at a glance that the program is still running and has not hung.

Building it, part one: power to display

ASSEMBLY 1 OF 2

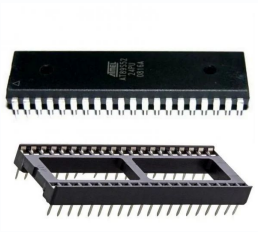
The board is built in stages, and the order matters: each stage is tested on its own before the next part is added, so a fault is always in the part you just fitted. Nothing here is soldered.

1. **Stage 0 — the bench.** A clean, dry, non-conductive surface, a 9 V adapter, and the parts sorted and counted against the bill of materials first. Counting the parts first saves an hour later.
2. **Stage 1 — power, alone.** The barrel jack, the protection diode, the 7805 regulator on its heatsink and its capacitors. This is built and measured before anything else is plugged in, because every other part depends on a clean 5 V.
3. **Stage 2 — the minimum system.** The chip, the 11.0592 MHz crystal with its two 33 pF capacitors, the reset circuit and the EA strap tied high. With only these fitted and no program at all, the chip is already a running computer.
4. **Stage 3 — the LED bank.** The latch, the pull-up resistor network, eight 470 ohm resistors and the eight LEDs. Program 01 is burned in at this point: if eight LEDs blink, the clock, the bus and the latch all work.
5. **Stage 4 — the switches.** The eight-way DIP switch, its pull-up network and the interrupt buttons. Program 03 turns this into the first two-way conversation: read a switch, light an LED.
6. **Stage 5 — the display.** The second latch, the digit driver transistors and the two-digit display. Program 04 counts 00 to 99, which proves the 5 ms timer interrupt is running.



Stages 0 to 1

Power first, on its own, measured with a meter before anything else is fitted.



Stage 2

Chip, crystal and reset. With only these three things the board is a working computer.



Stages 3 to 5

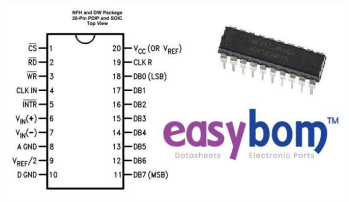
Outputs, then inputs, then the display: each stage is proved with its own program before the next begins.

Building it, part two: talking to the world

ASSEMBLY 2 OF 2

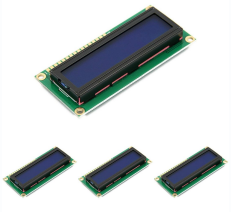
The second half of the build adds everything that lets the board communicate: with a person, through the display and the buzzer; with a PC, down a cable; and with the physical world, through the relay.

1. **Stage 6 — the serial link.** The MAX232 level converter and its four capacitors, then the nine-pin connector. The chip's serial port speaks 5 V logic; a PC serial port expects plus and minus 12 V. This chip does the translating, in both directions.
2. **Stage 7 — the temperature input.** The ADC0804, its reference divider, its resistor and capacitor clock, and the LM35 sensor. Program 10 is burned in and the reading is compared against a real thermometer.
3. **Stage 8 — the LCD.** The module on a removable header, its contrast potentiometer and its current-limiting resistor. Program 11 draws the temperature as a number and as a growing bar.
4. **Stage 9 — the relay and buzzer.** The driver transistor, the back-EMF protection diode across the coil, the relay itself, the jumper that frees the pin for the square-wave test, and the buzzer.
5. **Stage 10 — final checkout.** Program 13 is burned in and runs a self-test of every part in turn, printing PASS or naming the fault. Then program 12 goes in and stays in.



Stages 6 to 7

The serial link to the PC, then the converter and the sensor that give the board something to think about.



Stage 8

The LCD turns numbers into something a person can read without a PC attached.



Stage 9

The relay and buzzer: the point where the board stops being a display and starts doing something.

WHY BUILD IN THIS ORDER

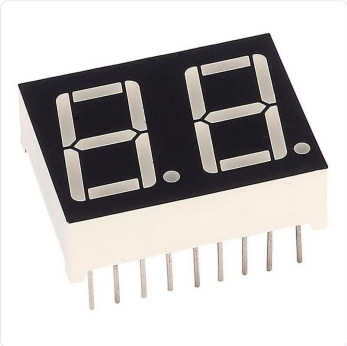
Power, then the chip, then outputs, then inputs, then communication, then the load. Every stage can be tested with the parts already fitted, so a fault is always in the last thing you added. Building it all at once and testing at the end is the single most common reason student boards do not work.

What happens when you press the button

WALKTHROUGH 1 OF 2

This is the question to be ready for, because it tests whether you understand the machine rather than just the diagram. Here is the emergency stop button, from fingertip to buzzer, in order, with times.

1. **You press PB1.** The button connects pin 12 of the chip, P3.2, straight to ground. The voltage on that pin falls from 5 V to 0 V.
2. **The chip notices, within a microsecond.** That pin is wired to external interrupt 0, and the code has configured it to trigger on a falling edge. The processor finishes the instruction it is in the middle of, saves its place, and jumps straight to the interrupt routine. Nothing else has to be polled or waited for.
3. **The routine does one thing.** It sets a flag called `emergency` to 1 and returns. Interrupt routines are kept this short on purpose; the actual work happens in the main loop.
4. **The main loop picks the flag up** on its very next pass, which is within a few hundred microseconds. From here on the emergency state wins every decision.
5. **The relay is dropped out.** Whatever the temperature is, the relay pin is driven low. The load it was switching is disconnected. This is the whole point of the button.
6. **The buzzer is driven on** and the red alarm LED lights, overriding the green or yellow that was showing.
7. **The LCD changes.** Line two is overwritten with EMERGENCY STOP. The temperature on line one keeps updating, so the reading is not lost.
8. **The next telemetry line** to the PC reports the zone as EMERGENCY with the relay off and the buzzer on, so the log shows exactly what happened and when.
9. **You press PB2.** That is interrupt 1. It clears the flag, the buzzer stops, and normal temperature control resumes immediately from the current reading.



Through all of this the two-digit display keeps showing the live temperature. The display is refreshed by a timer interrupt that the main loop can never block.

What happens when the room gets hot

WALKTHROUGH 2 OF 2

The second question worth having ready. This one crosses from the physical world into numbers and back out again, which is exactly what an embedded system is for. Suppose the air around the sensor warms to 45 degrees Celsius.

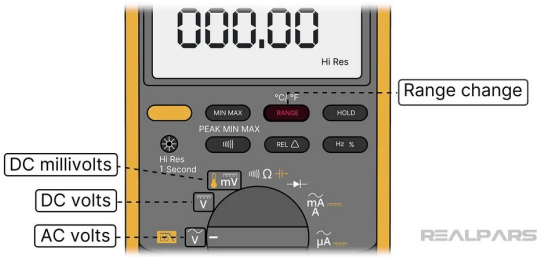
1. **The sensor feels it.** The LM35 is built to produce 10 millivolts for every degree. At 45 degrees its output leg carries 450 millivolts. That is a real, measurable voltage on a piece of wire.
2. **The converter is given a reference.** A trimmed divider holds the converter's reference pin at exactly 1.000 volt, which makes each of its 256 steps worth 7.8125 millivolts. This was chosen deliberately: with the default 5 V reference each step would be 1.95 degrees, and most of the scale would be wasted on temperatures the sensor can never see.
3. **The chip asks for a reading.** It pulses the write pin, the converter takes about 114 microseconds, then pulls its interrupt pin low to say the answer is ready. The chip pulses the read pin and the answer appears on the shared bus: 450 divided by 7.8125 is roughly 58.
4. **The code turns 58 into degrees.** It multiplies by 250 and divides by 32, which gives 453, meaning 45.3 degrees, carried throughout as tenths so that no decimal arithmetic is ever needed on a chip with no floating-point unit.
5. **The decision is three comparisons.** 453 is not below 400, so it is not the safe zone. It is below 700, so it is the warning zone. The relay is switched on, the buzzer stays off, the yellow LED lights.
6. **The outputs move.** The LED pattern is assembled as a set of bits and pushed through the latch. The display is given the number 45. The LCD prints T=45.3C on line one and WARNING FAN:ON on line two.
7. **One line goes upstairs.** Every second the serial port sends T=45.3C ADC=58 ZONE=WARNING FAN=1 BUZ=0, which a terminal on the PC can log to a file.
8. **And it starts again.** Cross 70 degrees and the red LED and the buzzer join in. This is why the heartbeat LED matters: it proves the loop is still turning even when nothing else changes.

> **THE SENTENCE THAT ANSWERS BOTH WALKTHROUGHS**
>
> A voltage becomes a number, the number becomes a decision, and the decision becomes a voltage again. That is what an embedded system is, and this board does it in about a hundred microseconds.

How we checked it was right

VERIFICATION

Building a board that lights up is not the same as building one that is correct. Every claim on this project is tied to a measurement with a number and a tolerance, taken with the two instruments below.



The multimeter answers: is the voltage what it should be? Supply rails, the trimmed reference, sensor output and current draw are all measured with it.

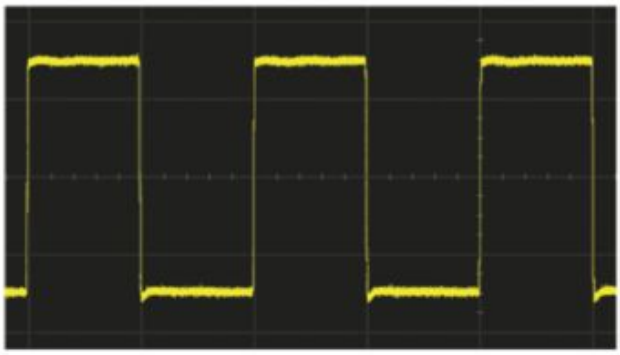


Figure 4. A square wave.

The oscilloscope answers: is the timing what it should be? The 1.00 kHz square wave, the serial bit width and the display refresh are all checked on it.

#	Check	How it is done	What should happen
T2	Supply rails	Meter on the regulator output, the chip's VCC pin and the converter's VCC pin	5.00 V, plus or minus 0.15 V, at all three
T3	LED timing	Burn program 01, time ten flashes with a stopwatch	Ten flashes in 10.0 seconds, plus or minus 0.3 s
T5	Display count	Burn program 04, time the count from 00 to 99	50 seconds, plus or minus 2 s
T6	One second timing	Burn program 05, time ten LED transitions	Ten transitions in 10.0 seconds, plus or minus 0.2 s
T7	Square wave	Scope on pin P3.4 with the relay jumper removed	1.00 kHz, within 20 Hz, at 50 percent duty
T9	Serial link	Short pins 2 and 3 of the connector and type characters	Every character comes back, none corrupted
T11	Temperature accuracy	Compare against a reference thermometer from 25 to 60 degrees	Within 1.5 degrees at every point
T13	Zone thresholds	Warm the sensor slowly and watch the LEDs	Green below 40.0, yellow from 40.0, red at 70.0 and above
T15	Emergency stop	Press PB1 while the board is running normally	Relay drops out, buzzer sounds, LCD shows EMERGENCY STOP
T18	Current draw	Ammeter in series with the 9 V supply, relay off	Below 200 mA
T19	Regulator heat	Run for fifteen minutes, then touch the heatsink	Warm but holdable, under about 60 degrees
T20	Supply rejection	Vary the input from 7 V to 12 V	5 V rail stays between 4.90 and 5.10 V, no reset

The numbers

RESULTS

These are the measured figures for the finished board. Each one is the answer to a test in the procedure, not an estimate, and each one comes with a short reason why it came out that way.

0.00 %
SERIAL BAUD ERROR AT 9600

1.00 kHz
TIMER OUTPUT, +0.17 % ERROR

100 Hz
DISPLAY REFRESH, PER DIGIT

0.78 °C
TEMPERATURE RESOLUTION PER STEP

157 mA
BOARD CURRENT, RELAY OFF

3.4 KB
LARGEST PROGRAM OF 8 KB FLASH

Measured	Value	Why it came out that way
Baud error	0.00 %	11.0592 MHz divides exactly by 384 and by 9600, so the timer reload is a whole number, 0xFD.
Square wave	1.00 kHz, +0.17 %	Reload value 0xFE34 with a 1.0851 microsecond machine cycle. The residual error is the interrupt response time.
Display refresh	100 Hz	A 5 ms timer interrupt refreshes one digit, so a full pair takes 10 ms. Fast enough that the eye sees both as solid.
ADC resolution	7.8125 mV, 0.78 °C	The reference pin is trimmed to 1.000 V rather than left at 5 V, which spends the whole scale on 0 to 200 degrees.
Board current	157 mA, 274 mA peak	Eight LEDs at 7 mA, the display, the LCD and the logic. The peak is with the relay coil energised.
Program size	3.4 KB, 30 B RAM	Largest of the thirteen. Memory was never the constraint on this board; pins were.
Component count	80 parts	Through-hole, hand placeable, no printed circuit board and no surface-mount work.
Cost	about INR 985	Per board, at single-unit prices. A full setup with adapter and programmer is about INR 1,715.

IF YOU ARE ASKED WHICH NUMBER YOU ARE PROUDEST OF

The baud error of 0.00 per cent. It is not luck: it is the direct result of choosing an 11.0592 MHz crystal instead of a rounder 12 MHz one, and it is the clearest example in the project of a hardware choice made in order to make the software simple.

Plain-English glossary

REFERENCE

Every technical term used in this guide, in the order it is likely to come up. If you can say each of these out loud in one plain sentence, you can present any part of this project.

Term	What it means here
Microcontroller	A complete small computer in one chip: processor, program memory, working memory and input/output pins. This one is an Atmel AT89S52.
8051	The family of chip design this belongs to, created in 1980 and still made today. Code written for one 8051 generally runs on another.
Pin	One of the forty metal legs of the chip. Software affects the world only by setting a pin to 5 V or 0 V, or reading which it is. They are handled in four groups of eight, called ports P0 to P3, giving thirty-two in total, every one of them shared between several jobs.
.hex file	The compiled program, written as a text file of machine bytes in Intel hex format. This is the file the programmer burns into the chip.
Flash memory	The 8 KB of memory inside the chip that holds the program. It keeps its contents with the power off and can be rewritten hundreds of times. Variables live in 256 bytes of RAM, which is wiped at every power-off.
Compiler	The tool that turns the C you typed into machine bytes the chip can execute. Here it is the C51 compiler inside Keil uVision.
Programmer	The small USB board that copies the .hex file from the PC into the chip's flash memory.
Crystal	The 11.0592 MHz part that sets the chip's heartbeat. Every instruction and every delay is counted in these ticks.
Machine cycle	1.0851 microseconds at this clock speed. Twelve crystal ticks make one machine cycle, and most instructions take one or two of them.
Bus	A set of wires shared by several devices. Here, eight wires carry data to the LEDs, the display, the LCD and the converter in turn.
Latch	A chip that copies the eight values on its input and keeps holding them on its output until told to change. This is what makes sharing a bus possible.
Strobe / gate	A single pin that tells one latch to open and capture what is on the bus right now. Four gates, four devices, one bus.
Pull-up resistor	A resistor that holds a pin gently at 5 V so that it never floats at an undefined voltage. The switches pull it down to 0 V against it.
Current-limiting resistor	The 470 ohm resistor in series with each LED. Without it the LED would draw unlimited current and burn out.
ADC	Analogue-to-digital converter. Measures a voltage and reports it as a whole number, here from 0 to 255.
Reference voltage	The voltage the converter measures against. Trimmed to 1.000 V here, which sets the step size and therefore the accuracy.
Interrupt	A signal that makes the chip stop what it is doing immediately and run a short routine, called an ISR. Kept short so the main loop is never delayed. Used here for the buttons and for the display refresh.
Timer	A hardware counter inside the chip that can be set to interrupt after an exact number of microseconds. Used for the 5 ms display tick and the 1.00 kHz output.
UART	The chip's built-in serial port. Sends and receives bytes one bit at a time down a single wire in each direction.
Baud rate	How many bits per second the serial link sends. 9600 here, meaning one character every 1.04 milliseconds.
MAX232	The chip that converts the 0 V and 5 V of the chip's serial port into the plus and minus 12 V a PC serial port expects.
Relay	An electrically operated switch. 5 V on a chip pin lets it switch a mains-voltage load. It clicks audibly when it changes.
Back-EMF diode	The diode across the relay coil. When the coil switches off it generates a large voltage spike, and this diode absorbs it before it can damage the driver.
Regulator	The 7805, which turns the 9 V from the adapter into a steady 5 V. The extra 4 V leaves as heat, hence the heatsink.

How to present this project

DELIVERY

Keep a copy of this page in front of you during a viva. The difference between a good demonstration and a bad one is almost always having the short version ready.

The thirty-second version

> **SAY THIS FIRST**
>
> This board monitors temperature using an 8051 microcontroller and controls a relay and a buzzer based on three zones. The interesting part is that the design needed 54 connections and the chip only has 32 pins available, so I shared one eight-bit bus between four devices using a latch and a gate pin for each. The software is thirteen small Keil C programs, the largest 3.4 kilobytes, and everything it does is read a sensor, convert it to a number, decide, and move an output. Shall I start the board?

Then the two-minute version, in four beats

1. **Show the board, not the slides.** Point at the parts in order: chip, crystal, LEDs, display, LCD, sensor, relay. Say what each does in one sentence. Thirty seconds.
2. **State the constraint.** 54 wires needed, 32 pins available. Name the two options you rejected and why. Thirty seconds.
3. **Explain the fix.** One shared bus, four gates, one open at a time. Draw it on paper if you can. This is the part that earns the marks. Forty seconds.
4. **Demonstrate.** Program 12 running. Warm the sensor with a finger and watch the zone change. Press the emergency button. Show the telemetry on the PC. Twenty seconds.

Five questions you will be asked

Question	Answer in one or two sentences
Why not use a bigger chip?	It would need a printed circuit board and surface-mount soldering, and it would stop being a breadboard project. The constraint was the point.
Why not add port expanders?	Two more chips and about 250 rupees, and every update becomes a slow serial transaction. Sharing cost nothing but a few microseconds.
Why 11.0592 MHz and not 12?	Because it divides exactly by 9600, so the serial baud rate has literally zero error. With 12 MHz the error is over 8 per cent and the link fails.
How accurate is the temperature?	0.78 degrees per step, and within 1.5 degrees of a reference thermometer across 25 to 60 degrees. That comes from trimming the converter reference to 1.000 V.
What would you do differently?	Move to a printed circuit board with a ground plane, and use a digital-output sensor to remove the converter and its reference entirely.

Where every file lives

Folder	What is in it
firmware/src	The thirteen C programs, 01 to 13
firmware/inc/board.h	Every pin, every constant and every shared function, in one place
firmware/keil	A ready-to-open Keil uVision project for each program, for both the AT89S52 and the AT89C51
hardware/BOM.csv	All 80 parts with quantities and prices
hardware/SCHEMATIC.md	Every connection, written out pin by pin
hardware/ASSEMBLY.md	The ten build stages in full detail
hardware/TESTING.md	All twenty tests with their expected results
docs/	The seven-slide review deck and this guide